关注微信公众号：Java技术栈，干货及时推送。

昵称： Java技术栈
园龄： 8年3个月
粉丝： 566
关注： 42
+加关注

搜索
找找看

随笔分类 (229)
Eclipse(1)
Git(2)
Intellij IDEA(3)
Java(80)
Linux(1)
Maven(4)
Redis(9)
Spring(3)
Spring Boot(56)
Spring Cloud(17)
Tomcat(1)
程序人生(17)
工具(4)
后端(14)
技术动态(4)
更多

阅读排行榜
1. Java 中初始化 List 集合的 6 种方式!(85226)
2. Redis 常用操作命令，非常详细！(81272)
3. 常用的 7 款 MySQL 客户端工具，你值

榨干 Claude Code 的 16 个实用小技巧（高端玩法，建议收藏！）

大家好，我是R哥。

这几年 AI 编程工具越来越火，尤其是 Cursor、Claude Code 这种终端级 AI 编程助手，用好了真的能让程序员开发效率直接起飞。

但很多同学用 AI 写代码，常常卡在 “不会提问”、“不会拆需求”、“不会控制上下文”，只会傻傻提问，很多实用技巧都不会，结果总是写不出自己想要的效果。

今天这篇笔记，R哥就手把手带你盘一盘：**如何高效用 Claude Code 搞定开发、提效撸活，告别低效体力活！**（内容干货，建议收藏慢慢看～）

Claude Code 的安装和使用先看这篇：

> 自从用上了 Claude Code，才发现 Cursor 和 Gemini Cli 都是弱智。。（保姆级安装和使用教程分享）

1、把你的需求说具体点

别再说：“**修复这个漏洞**” 这么笼统的指令：尽量把你的需求说具体点。

你可以这样和它说：

> 修复用户登录时不输入密码出现的空指针错误

2、把复杂的需求分步执行

如果是小任务/模块，可以一次性把需求发给 AI，一次性出结果，整体效率更高。

但是，如果是大需求，实现流程比较长的那种，建议把复杂的任务拆解成小步骤，比如：

- 给用户 API 创建一个新接口；
- 给请求的字段添加必要的验证；
- 编写这个接口的测试用例；
- ...

因为 AI 上下文都有限制，代码量/记忆太长可能输出不全、甚至被截断，还是分步最安全，每一步完成后你都可以先 review/测试，再让 AI 执行下一步。

> 像上面的示例，其实并不复杂，可以一次性发给 Claude Code，AI 能整体考虑代码结构和风格，减少重复解释和沟通成本。

3、先理解项目代码再开动

在修改代码之前，先让 Claude 理解你的代码，比如：

- 分析一下数据库表结构；
- 这个应用中的错误是如何处理的？
- ...

修改前，先让 AI 理解你业务和代码，这样才能更精准、高效地辅助你开发和优化。

4、学会用快捷键，节省时间

比如：

- 输入 / 查看所有斜杠命令；

2
升级成为会员

得拥有！(72479)

4. Spring Boot Tomcat配置详解(50187)

5. 面试官问线程安全的List，看完再也不怕了！(38569)

6. 干货 | Java中获取类名的3种方法！(30606)

7. 盘点 35 个 Apache 顶级项目，我拜服了...(28030)

8. 榨干 Claude Code 的 16 个实用小技巧（高端玩法，建议收藏！）(27909)

9. Redis 到底是单线程还是多线程？我要吊打面试官！(27510)

10. Spring Cloud Gateway VS Zuul 比较，怎么选择？(26090)

评论排行榜

1. 面试了一个 39 岁程序员，我有点慌.....(114)

2. 同事写了一个疯狂的类构造器，我要疯了，Builder 模式都不会么？！(38)

3. 面试了一个 39 岁程序员后，我被骂了.....(38)

4. if else 太多？看我用 Java 8 轻松干掉！(23)

5. 去了两家外包公司，颠覆了我的认知！(17)

推荐排行榜

1. 同事写了一个疯狂的类构造器，我要疯了，Builder 模式都不会么？！(21)

2. 面试了一个 39 岁程序员，我有点慌.....(20)

3. Redis 常用操作命令，非常详细！(20)

4. Spring事务失效的 8 大原因，这次可以吊打面试官了！(19)

5. Java 中初始化 List 集合的 6 种方式!(14)

最新评论

1. Re:用上了 Claude Code，才发现 Cursor 和 Gemini Cli 都是弱智。。（保姆级安装和使用教程分享）

@xiang/kf 再换个节点就好了 我换了一个印度的就行了 很神奇了。。...

--tofuliuo

2. Re:榨干 Claude Code 的 16 个实用小技巧（高端玩法，建议收藏！）

- 使用上下方向键查看命令历史；
- 使用 Tab 键进行命令快速补全；
- 使用 `Option + Enter` 换行；
- 使用 `Ctrl + C` 退出终端等等。

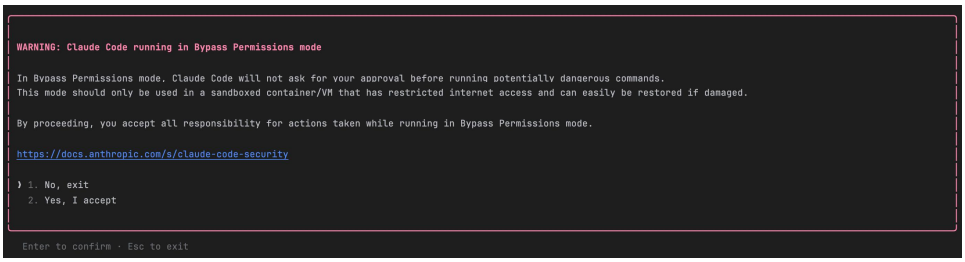
5、使用免授权模式

你是不是经常遇到 Claude Code 干活干一半，停下来让你授权？不授权就卡在那里，虽然可以在当前会话可以设置不再询问，但权限又有好几种，每种权限都要来问一下，严重影响效率。

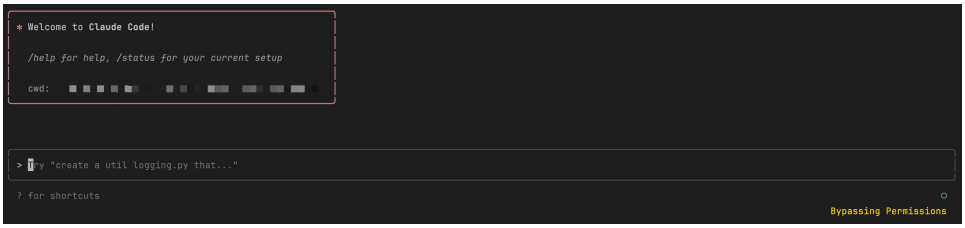
其实，启动 claude 时有一个参数：

```
claude --dangerously-skip-permissions
```

带上这个参数启动时，Claude Code 会出现警告提示：



你需要点确认（Yes）才能开启 **Bypassing Permissions** 模式，开启此模式后，终端下面会出现黄色的 `Bypassing Permissions` 模式提示：



开启 **Bypassing Permissions** 模式后，后面所有操作就都不需要你授权了，Claude Code 哐当就把所有活干了。

每次都输入 `claude --dangerously-skip-permissions` 太麻烦了，还可能输错，可以考虑他那一个 alias 别名：

> alias claude='claude --dangerously-skip-permissions'

这只是临时生效，永久生效可以把它放到个人环境配置文件中，然后 source 生效一下。

6、激活深度思考模式

在 Claude Code 中，可以使用 “**think**” 这个词来激活深度思考模式，包括以下几种级别：

> “think” < “think hard” < “think harder” < “ultrathink”

使用这些深度思考，会直接对应系统中不同级别的思考预算，每一级都会逐步增加 Claude 可用的思考预算，毫无疑问，使用 `ultrathink` 是最费钱的，也能发挥它的最大潜能。

如果你是订阅的是 Max 套餐，可以考虑使用 `ultrathink` 模式，不然就得小心你的钱包。

比如我来测试一下：

> 1+1=? ultrahink

2

升级成为会员

谢谢大佬教程

--喷气

3. Re:用上了 Claude Code，才发现 Cursor 和 Gemini Cli 都是弱智。。（保姆级安装和使用教程分享）

启动没有代理问题吗？gemini cli我能用，但是claude code启动说我的地区不支持

--xiang/kf

4. Re:领导：谁再用 Redis 实现过期订单关闭，立马滚蛋！

一直轮询数据库，都用好几年了，没什么问题

--Flamings

5. Re:Spring Boot 分片上传、断点续传、大文件上传、秒传，应有尽有，建议收藏！！

我实现过，炒鸡简单

--老顽皮

```
> /cost
  └ Total cost:            $0.0619
    Total duration (API):  7.2s
    Total duration (wall): 2m 7.8s
    Total code changes:    0 lines added, 0 lines removed
    Token usage by model:
        claude-3-5-haiku:  217 input, 7 output, 0 cache read, 0 cache write
           claude-sonnet:  4 input, 7 output, 0 cache read, 16.4k cache write
```

一个 1+1 计算耗费了 0.06 美元，大概不到 5 毛 RMB。。

7、输错指令，随时打断它工作

如果在 Claude Code 工作时，有时候可能给的命令描述的不对，如果你想让它停止，只需要按 `ESC` 键即可：

```
> 总结一下这个项目
  └ Interrupted by user

 >

 ? for shortcuts
```

终端上面会显示被用户打断。

8、发送图片处理

Claude Code 可以发送图片并进行处理，在命令行中，把图片和提示词发过去，让它更好的理解你的意图。

如图所示：

```
> [Image #4] 这个按钮样式错乱了，请修复
```

> 注意，在 Mac 中粘贴图片不是使用 command + v，而是使用 **ctrl + v** 快捷键。

你还可以发送以下命令：

- 这个图片显示了什么？
- 这是错误的截图，是什么原因导致的？
- 请根据这个图片的设计模型设计网页

9、恢复历史会话

非交互模式

Claude Code 提供两个选项来恢复之前的对话：

- `claude --continue` 或者 `claude -c` ：自动继续最近的对话，无需任何提示。
- `claude --resume` 或者 `claude -r` ：显示历史对话选择器；

这两个带参数的命令需要在「**非交互模式**」下进行，也就是还没有进入 Claude Code。

交互模式

2

升级成为会员

如果你已经进入了 Claude Code 会话，想恢复到之前的哪个历史会话，可以使用 `/resume` 命令恢复历史会话：

```
    Modified    Created     # Messages Summary
❯ 1. 2m ago     57m ago            69 mybatis-plus 模块请添加一个页面用添加用户吧，再添加一个页面用来搜索用户
  2. 6m ago     57m ago            69 mybatis-plus 模块请添加一个页面用添加用户吧，再添加一个页面用来搜索用户
  3. 8m ago     57m ago            69 mybatis-plus 模块请添加一个页面用添加用户吧，再添加一个页面用来搜索用户
  4. 12m ago    1h ago              3 Caveat: The messages below were generated by ...
  5. 1h ago     1h ago              3 IntelliJ IDEA Connection and Local Command Execution
  6. 1h ago     1h ago              3 VS Code Connection and Local Command Execution
  7. 1h ago     1h ago              3 Cost Analysis for Local Command Execution
  8. 2h ago     2h ago             54 MyBatis-Plus: Add User Creation Interface Endpoint
  9. 3h ago     4h ago             38 Spring Boot Best Practices Project Tech Stack Overview
  10. 3h ago     3h ago              3 Local Command Help Request, No Output
  11. 3h ago     3h ago              5 Screen Cleared, Ready for Spring Boot Development
  12. 3h ago     3h ago              6 Spring Boot Project Setup and Configuration Assistance
```

上下方向键选中一条记录可以恢复会话。

10、记忆管理

记忆文件介绍

Claude Code 提供三种记忆位置，每种都有不同用途：

记忆类型	文件位置	用途说明	使用示例
项目记忆（共享）	`./CLAUDE.md`	项目团队共享的指令	项目架构、编码规范、常用工作流程
用户记忆（全局）	`~/.claude/CLAUDE.md`	用于所有项目的个人偏好设置	代码风格偏好、个人工具快捷方式
项目记忆（本地）	`./CLAUDE.local.md`	项目的个人偏好设置（已废弃）	你的沙箱地址、测试数据偏好等

其中， `CLAUDE.md` 文件是 Claude Code 自动读取的记忆文件，类似于 Cursor 中 `rules` 规则文件，但比它要更强大，它可以为 Claude 提供更多项目相关的上下文信息，如：

- 常用的 bash 命令
- 核心文件和工具函数
- 代码风格指南
- 测试说明
- 代码库规范
- 开发环境设置
- 更多希望 Claude 记住的信息等等

当 Claude Code 启动时，以上所有记忆文件会自动加载到运行环境中。

可以在多个位置放置 `CLAUDE.md` 文件，Claude Code 会递归读取这些文件，从当前工作目录开始，向上递归到根目录，读取找到的任何 `CLAUDE.md` 文件。

编辑记忆文件

在会话期间使用 `/memory` 斜杠命令，可以在系统编辑器中打开记忆文件：

```
Select memory to edit:

❯ 1. Project memory          Checked in at ./CLAUDE.md
  2. Project memory (local)  Gitignored in ./CLAUDE.local.md
  3. User memory             Saved in ~/.claude/CLAUDE.md

55 memories in ./CLAUDE.md
```

2

升级成为会员

选择一个记忆文件回车进行编辑，其中第一个就是使用 `/init` 初始化命令生成的，第二个已废弃，第三个是用户级记忆文件。

比如我们可以修改第三个用户级记忆文件：

> 每次请用中文回答我。

这样设置记忆后，后续所有项目的交互就都是中文回答的了。

11、和 Git 进行交互

在 Claude Code 中，我们的 Git 操作就可以变得对话形式，不用记住一些繁琐的命令。

> 我修改了哪些文件

```
> 我修改了哪些文件

● Bash(git status)
  └ On branch master
    Your branch is up to date with 'origin/master'.

    … +5 lines (ctrl+r to expand)

● 您创建了一个新文件：

  - CLAUDE.md - 新创建的项目文档文件，包含了项目概述、开发命令、架构结构等信息，并已转换为中文友好的内容

  该文件目前处于未跟踪状态，需要使用 git add CLAUDE.md 来添加到版本控制中。
```

> 用合理描述性信息提交我的更改

```
> 用合理描述性信息提交我的更改

● Bash(git add CLAUDE.md)
  └ (No content)

● Bash(git commit -m "添加项目文档 CLAUDE.md…)
  └ [master bc92e69] 添加项目文档 CLAUDE.md
     1 file changed, 92 insertions(+)
     create mode 100644 CLAUDE.md

● Bash(git status)
  └ On branch master
    Your branch is ahead of 'origin/master' by 1 commit.
      (use "git push" to publish your local commits)
    … +2 lines (ctrl+r to expand)

● 已成功提交您的更改！

  提交信息：
  - 添加项目文档 CLAUDE.md
  - 创建中文友好的项目文档，包含微信小程序架构说明
  - 详细说明开发命令、目录结构和云函数架构
  - 提供代码模式和配置文件说明，便于开发者快速上手

  提交ID: bc92e69状态：工作目录干净，本地分支领先远程分支1个提交
```

> 推送本分支到远程

```
> 推送本分支到远程

● Bash(git push)
  └ To github.com:[redacted].git
      b397b9c..bc92e69  master -> master

● 已成功推送到远程仓库！本地 master 分支的更改已同步到 github.com:[redacted].git。
```

> 创建一个新分支:feature/test

2

升级成为会员

```
> 创建一个新分支:feature/test

● Bash(git checkout -b feature/test)
  └ Switched to a new branch 'feature/test'

● 已成功创建并切换到新分支 feature/test。
```

> 删除本分支并切换到master分支

```
> 删除本分支并切换到master分支

● Bash(git checkout master)
  └ Your branch is up to date with 'origin/master'.
    Switched to branch 'master'

● Bash(git branch -d feature/test)
  └ Deleted branch feature/test (was bc92e69).

● 已成功切换到 master 分支并删除了 feature/test 分支。
```

> 显示最近3次提交中所有文件列表

```
● 最近3次提交中的所有文件列表:

  bc92e69 - 添加项目文档 CLAUDE.md
  - CLAUDE.md

  b397b9c - 1
  -
  -
  -

  85d2fd1 - 更新
  -
  -
  -
  -
  -
  -
```

很方便快捷吧，使用自然语言和 Git 进行交互吧！

12、和 Linux 交互

因为 Claude Code 是终端形式使用嘛，所以我们也可以把它当作 Linux 智能助手用，无需记住繁杂的 Linux 命令。

在交互模式下：

> 列出行数最多的前3个.java文件

```
> 列出行数最多的前3个.java文件

● Bash(find . -name "*.java" -exec wc -l {} \; | sort -nr | head -3)
  └      200 ./spring-boot-redis/src/main/java/cn/javastack/springboot/redis/service/RedisOptService.java
         140 ./spring-boot-web/src/main/java/cn/javastack/springboot/web/config/WebConfig.java
         134 ./spring-boot-mapstruct/target/generated-sources/annotations/cn/javastack/springboot/mapstruct/struct/UserNestedStructImpl.java

● 前3个行数最多的.java文件:

  1. RedisOptService.java - 200行
  2. WebConfig.java - 140行
  3. UserNestedStructImpl.java - 134行
```

从执行信息可以看到它正在执行的命令，这么复杂的命令一般人是很难记住的。

在非交互模式下：

> claude -p "列出行数最多的前3个.java文件"

2

升级成为会员

```
📝 Press Enter to continue...

根据搜索结果，行数最多的前3个Java文件是：

1. **RedisOptService.java** (200行) - `./spring-boot-redis/src/main/java/cn/javastack/springboot/redis/service/RedisOptService.java`
2. **WebConfig.java** (140行) - `./spring-boot-web/src/main/java/cn/javastack/springboot/web/config/WebConfig.java`
3. **UserNestedStructImpl.java** (134行) - `./spring-boot-mapstruct/target/generated-sources/annotations/cn/javastack/springboot/mapstruct/struct/UserNestedStructImpl.java`

需要注意的是，第3个文件是在target目录下的生成文件，如果你只想看源代码文件，我可以排除target目录重新搜索。
```

这是执行一次命令，它会列出满足条件的文件后退出 Claude Code 交互模式。

13、模型切换

Claude Code 目前支持 **Claude Opus 与 Claude Sonnet 4** 两个模型的灵活切换，使用 `/model` 命令进行切换：

```
Select Model
Switch between Claude models. Applies to this session and future Claude Code sessions. For custom model names, specify with --model.

❯ 1. Default (recommended)  Use the default model (currently Sonnet 4) · $3/$15 per Mtok✔
  2. Opus                   Opus 4 for complex tasks · $15/$75 per Mtok

Enter to confirm · Esc to exit
```

默认为 **Claude Sonnet 4**，可以切换到 Claude Opus，不过个人感觉没必要，**强烈推荐使用 Claude Sonnet 4，其使用体验与Claude Opus 并没有明显差别，但计费倍率仅为其 1/5**，土豪请忽略。

> 注意，**只有 Max 用户才支持 Claude Opus 并支持切换**，Pro 用户只支持 Claude Sonnet 4。

14、查看消耗情况

使用 `/cost` 命令查看当前会话使用情况：

```
> /cost
  └ Total cost:            $1.23
    Total duration (API):  11m 45.0s
    Total duration (wall): 1h 2m 0.3s
    Total code changes:    620 lines added, 109 lines removed
    Token usage by model:
        claude-3-5-haiku:  4.1k input, 1.0k output, 0 cache read, 0 cache write
           claude-sonnet:  390 input, 15.3k output, 542.2k cache read, 220.7k cache write
```

如图，显示我当前会话已经消耗了 1.23 美金。

官方查看消耗，但过于笼统，不够直观，推荐使用 `ccusage` 工具来查看。

安装 ccusage：

```
sudo npm install -g ccusage
```

```
WARN  Fetching latest model pricing from LiteLLM...

ℹ Loaded pricing for 1140 models

╭────────────────────────────────────────────╮
│                                            │
│  Claude Code Token Usage Report - Daily    │
│                                            │
╰────────────────────────────────────────────╯
```

Date	Models	Input	Output	Cache Create	Cache Read	Total Tokens	Cost (USD)
2025-07-01	- sonnet-4	3,584	13,656	1,607,923	6,722,452	8,347,615	$8.26
2025-07-02	- sonnet-4	4	5	16,424	0	16,433	$0.06
Total		3,588	13,661	1,624,347	6,722,452	8,364,048	$8.32

如果要查看自某天开始的消耗：

2

升级成为会员

> ccusage -s 20250701

如果要实时查看消耗：

> ccusage blocks --live

Claude Pro / Max 订阅用户可以不用理会消耗，它是按月计费的，不是按使用量计费的，如下面所示：

```
> /cost
  └ With your Claude Pro subscription, no need to monitor cost — your
    subscription includes Claude Code usage
```

超过了使用量就会变得不可用：

```
  └ Claude usage limit reached. Your limit will reset at 8pm
    (Asia/Shanghai).

      • /upgrade to increase your usage limit.

  >

  ? for shortcuts                                   Bypassing Permissions
```

需要等待，到了指定时间才能恢复。

15、上下文压缩

Claude Code 提供了一个 `/compact` 压缩命令：

```
> /compact <optional custom summarization instructions>

/compact     Clear conversation history but keep a summary in context. Optional: /compact [instructions for summarization]
```

它会清除对话历史记录，但保留上下文中的摘要。

这样做的好处是：

- **减少对话上下文大小**：当对话历史变得很长时，使用 `/compact` 可以压缩对话内容，减少令牌使用量。
- **手动压缩控制**：虽然 Claude Code 默认在上下文超过 95% 容量时自动压缩（可通过 `/config` 开启/关闭自动压缩），但你可以使用 `/compact` 手动触发压缩。

如图所示：

```
* Navigating… (232s · ⚒ 8.3k tokens · esc to interrupt)

  >

  ? for shortcuts                                   Bypassing Permissions
                                                    Context left until auto-compact: 39%
```

当剩余上下文空间不多时，右下角会显示自动压缩的剩余百分比，当达到 0% 时就会进行压缩。

所以，为了有效管理成本和性能：

- 建议在上下文变大时定期使用 `/compact` 手动进行压缩；
- 定时使用 `clear` 命令重置上下文；
- 分解复杂任务或者把需求尽量具体化；

当然，土豪，请略过。

2

升级成为会员

16、自定义快捷命令

使用介绍

Claude Code 支持自定义命令，你可以创建一些命令来快速执行特定的提示或任务，比如：

- 分析这个项目的性能，并提出三个具体的优化建议。
- 用合理描述性信息提交所有变更文件，然后推送到远程仓库。
- ...

这样一些常用的操作就不需要写一堆文字了，用自定义命令即可。

自定义命令语法：

```
/<prefix>:<command-name> [arguments]
```

自定义命令解读：

- 命令分为用户级和项目级；
- 用户级命令所有项目都能用，项目级命令只有当前项目可以用；
- 用户级命令放在个人 `~/.claude/commands` 目录下，而项目级命令放在当前项目 `.claude/commands` 目录下；
- 使用命令时，用户级命令以 `/user:` 为前缀，项目级命令 `/project:` 为前缀，后面跟的是命令文件名称，可级联；
- 命令文件支持使用 `$ARGUMENTS` 参数占位符，在命令后面带上参数，如： `/project:test 123` 它会用 123 替换命令文件中的 `$ARGUMENTS` 标记。

比如如果有 `.claude/commands/frontend/component.md` 自定义命令，使用方法 就是：`/project:frontend:component` 。

实战应用

项目级命令

在当前项目创建自定义命令目录：

> mkdir -p .claude/commands

创建一个项目级优化命令：

> echo "分析这个项目的性能，并提出三个具体的优化建议。" > .claude/commands/optimize.md

在 Claude Code 中使用自定义命令：

> /project:optimize

2

升级成为会员

```
> /project:optimize

● Update Todos
  └ ☐ 分析项目整体架构和模块结构
    ☐ 识别性能瓶颈和问题代码
    ☐ 分析数据库操作和查询效率
    ☐ 提出三个具体的优化建议

● Update Todos
  └ ☐ 分析项目整体架构和模块结构
    ☐ 识别性能瓶颈和问题代码
    ☐ 分析数据库操作和查询效率
    ☐ 提出三个具体的优化建议

● Task(分析项目架构和性能)
  └ Done (27 tool uses · 38.0k tokens · 2m 17.7s)

● Update Todos
  └ ☒ 分析项目整体架构和模块结构
    ☒ 识别性能瓶颈和问题代码
    ☒ 分析数据库操作和查询效率
    ☐ 提出三个具体的优化建议
```

没想到它还自己把任务细化了，从分析项目整体架构、性能瓶颈、数据库操作，然后再提出优化建议。

用户级命令

在个人目录下创建自定义命令目录：

```
mkdir -p ~/.claude/commands
```

创建一个项目级优化命令：

```
echo "用合理描述性信息提交所有变更文件，然后推送到远程仓库。" > ~/.claude/commands/push.md
```

在 Claude Code 中使用自定义命令：

```
/user:push
```

2

升级成为会员

```
> /user:push

● Bash(git status)
  └ On branch master
    Your branch is up to date with 'origin/master'.

    … +5 lines (ctrl+r to expand)

● Bash(git diff)
  └ (No content)

● Bash(git log --oneline -3)
  └ bc92e69 添加项目文档 CLAUDE.md
    b397b9c 1
    85d2fd1 更新

● Bash(git add .claude/)
  └ (No content)

● Bash(git commit -m "添加Claude命令配置目录…)
  └ [master d3c6a53] 添加Claude命令配置目录
     1 file changed, 1 insertion(+)
     create mode 100644 .claude/commands/optimize.md

● Bash(git push)
  └ To github.com:javastacks/javastack-app.git
       bc92e69..d3c6a53  master -> master

● 已成功提交并推送所有变更！

  提交信息：
  - 添加Claude命令配置目录
  - 创建 .claude/commands/ 目录用于存储项目特定命令
  - 添加 optimize.md 命令文件，用于项目性能分析
  - 配置中文友好的命令模板，提升开发效率

  提交ID: d3c6a53状态: 已成功推送到远程仓库
```

nice~这样让 Git 交互也更简单了！

好了，这次的分享就到了～

以上就是我在实际使用 Claude Code 编程时的一些**高效技巧和避坑心得**，真的都是无保留实践总结。

AI 不会淘汰程序员，但不会用 AI 的除外，会用 AI 的程序员才有未来！

未完待续，接下来会继续分享下 Claude Code 心得体验、高级使用技巧，公众号持续分享 AI 实战干货，关注「**AI技术宅**」公众号和我一起学 AI。

> **版权声明：** 本文系公众号 "AI技术宅" 原创，转载、引用本文内容请注明出处，抄袭、洗稿一律投诉侵权，后果自负，并保留追究其法律责任的权利。

更多文章推荐：

1.Spring Boot 3.x 教程，太全了！

2.3,000+ 道 Java面试题及答案整理(最新版)

3.免费获取 IDEA 激活码的 7 种方式（最新版）

4.Java & DeepSeek & AI 学习资料分享

5.程序员精美简历模板分享

觉得不错，别忘了随手点赞+转发哦！

2

升级成为会员